

LECTURE MODULE • NATURAL LANGUAGE PROCESSING

Artificial Neural Networks

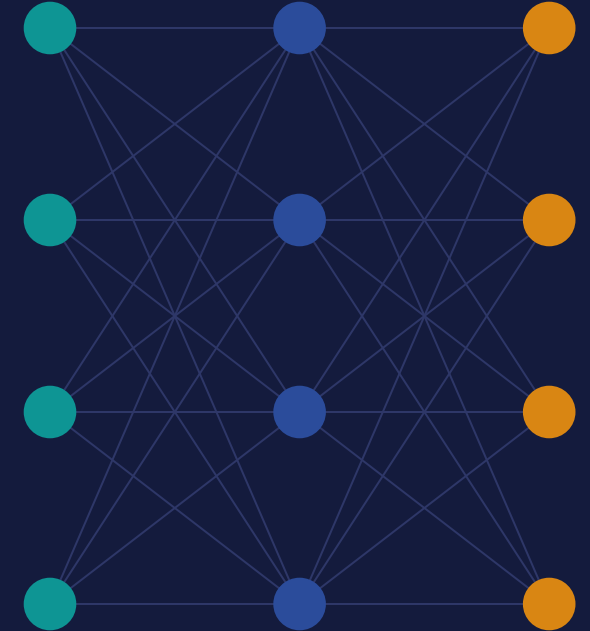
Feedforward Propagation, Backpropagation
and Activation Functions

A complete walkthrough with fully solved numerical examples

Undergraduate Level

3 Solved Examples

Practice Set Included



What we will cover

01

The Neuron

Biological inspiration, the perceptron, weights, bias, and why a single neuron is not enough.

02

Activation Functions

Step, Sigmoid, Tanh, ReLU, Leaky ReLU, ELU, GELU, Swish, Softmax — formulas, derivatives, trade-offs.

03

Feedforward Pass

Layer-by-layer computation, matrix form, and a fully solved forward pass.

04

Backpropagation

Chain rule, the four equations, and a complete hand-computed weight update.

05

NLP Application

A sentiment classifier worked end-to-end, plus softmax + cross-entropy for tagging.

Why neural networks took over NLP

The classical pipeline

- One-hot / bag-of-words: 50,000-dim sparse vectors
- "good" and "excellent" are orthogonal — zero similarity
- Features hand-engineered by linguists
- Feature combinations must be written by hand

The neural pipeline

- Dense embeddings: 300-dim learned vectors
- "good" and "excellent" end up close together
- Features learned automatically from data
- Hidden layers discover feature interactions

Every modern NLP model — word2vec, LSTMs, BERT, GPT — is a feedforward network trained by backpropagation.

A transformer is not a different kind of mathematics. It is the same forward pass and the same chain rule, arranged differently and repeated at scale. Master these two operations and the rest of the course is architecture, not arithmetic.

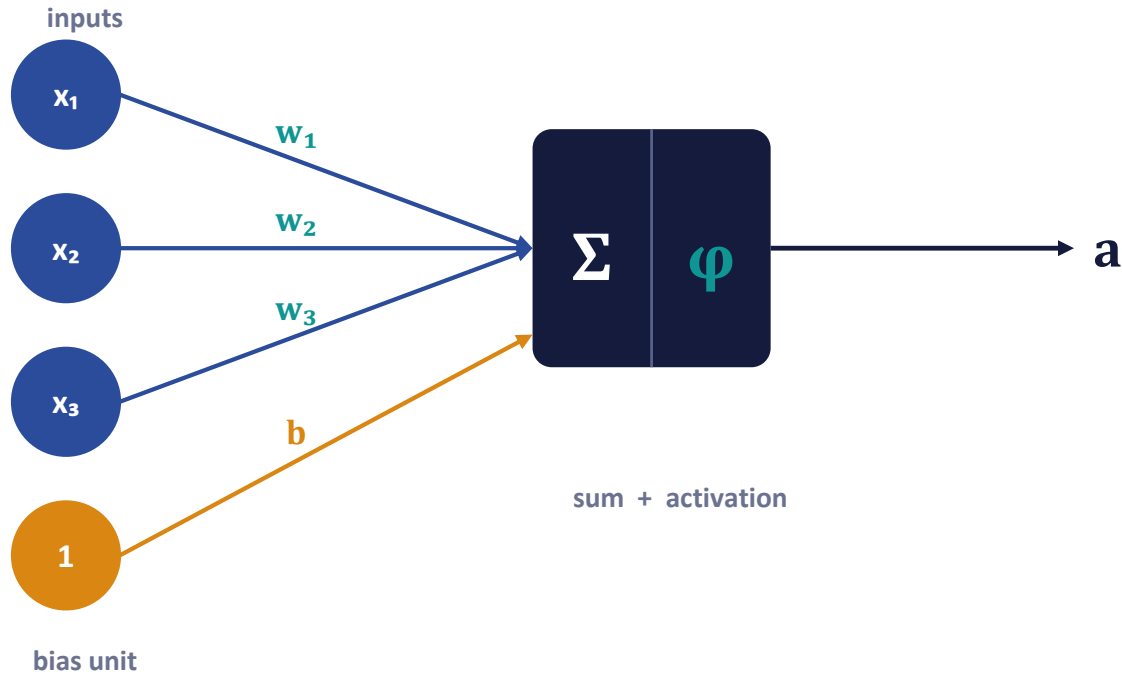
Task examples: sentiment classification · POS tagging · named entity recognition · machine translation · question answering

From biological neuron to artificial neuron

BIOLOGY	WHAT IT DOES	MATHEMATICAL COUNTERPART
● Dendrites	Receive signals from other neurons	Inputs $x_1, x_2, \dots, x_n$
● Synapse	Connection strength; can excite or inhibit	Weights $w_1, w_2, \dots, w_n$
● Cell body (soma)	Accumulates all incoming signals	Weighted sum $z = \sum w_i x_i + b$
● Firing threshold	Neuron fires only past a threshold	Bias b and activation $\phi(z)$
● Axon	Transmits the output onward	Output $a = \phi(z)$

The analogy is loose and historical. Do not over-read it — an artificial neuron is a differentiable function, not a simulation of biology.

Anatomy of a single artificial neuron



Written out in full: $a = \phi(w_1x_1 + w_2x_2 + w_3x_3 + b) = \phi(\mathbf{w} \cdot \mathbf{x} + b)$

Two-step computation

Step 1 $z = w_1x_1 + w_2x_2 + w_3x_3 + b$

Step 2 $a = \phi(z)$

What each part is for

- Weight w_i — how much input i matters. Learned.
- Bias b — shifts the threshold so the neuron can fire even when all inputs are zero. Learned.
- Σ — a linear combination; a plain dot product.
- ϕ — the non-linearity. Fixed by the designer, not learned.

One neuron, computed by hand

Given

$$x = [2.0 , \quad 3.0 , \quad -1.0]$$

$$w = [0.4 , \quad -0.5 , \quad 0.2] \quad b = 0.7$$

Step 1 — weighted sum

$$z = (0.4)(2.0) + (-0.5)(3.0) + (0.2)(-1.0) + 0.7$$

$$z = 0.8 + (-1.5) + (-0.2) + 0.7$$

$$z = -0.2$$

Step 2 — activation (sigmoid)

$$a = 1 / (1 + e^{(0.2)}) = 1 / 2.2214 = 0.4502$$

Reading the result

- z is negative, so the neuron is slightly inhibited.
- Sigmoid maps it to 0.4502 — just below the 0.5 decision point.
- If this were a spam classifier, output < 0.5 means “not spam”, but with very low confidence.
- Change b from 0.7 to 1.0 and z becomes +0.1, flipping the prediction. This is exactly what the bias is for.

Class exercise

Recompute a using tanh and using ReLU. Which one changes the sign of the output? Answer: tanh gives -0.1974 , ReLU gives 0.

The perceptron and the XOR problem

Perceptron (1958)

$$\hat{y} = 1 \text{ if } w \cdot x + b > 0$$

$$\hat{y} = 0 \text{ otherwise}$$

A single linear decision boundary. Can solve AND, OR, NOT.

AND — separable

$$(0, 0) \rightarrow 0 \quad (0, 1) \rightarrow 0$$

$$(1, 0) \rightarrow 0 \quad (1, 1) \rightarrow 1$$

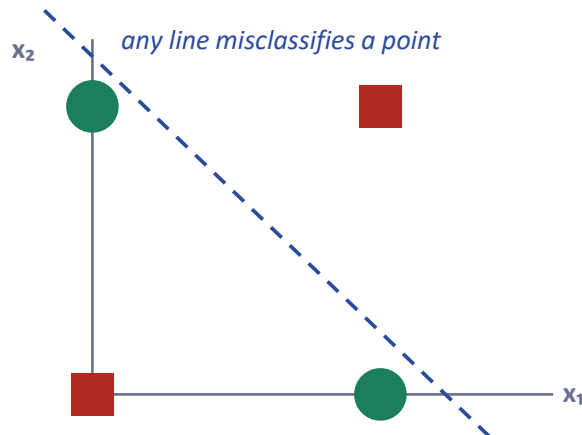
One straight line separates the single 1 from the three 0s.

XOR — NOT separable

$$(0, 0) \rightarrow 0 \quad (0, 1) \rightarrow 1$$

$$(1, 0) \rightarrow 1 \quad (1, 1) \rightarrow 0$$

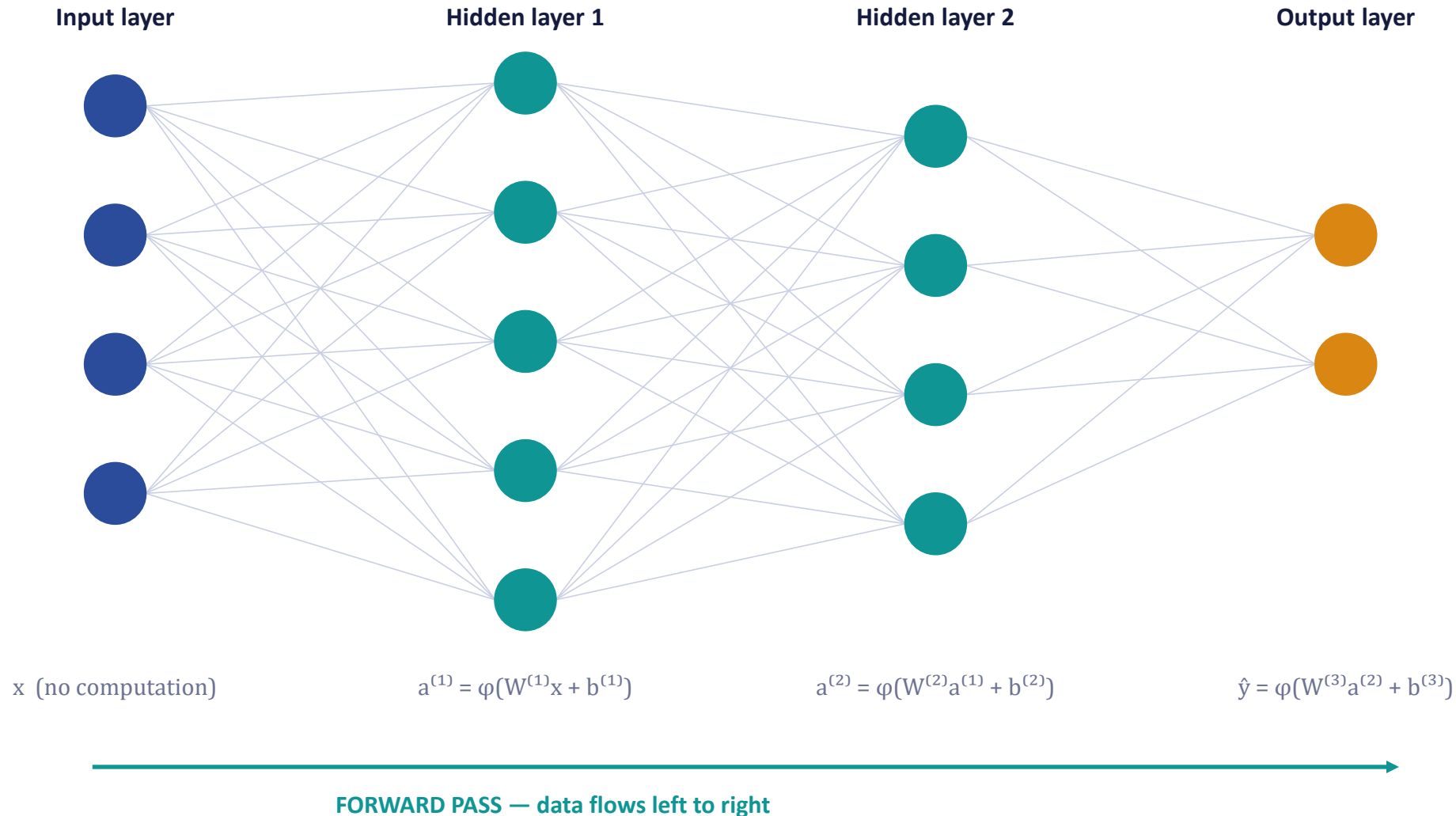
No single straight line can do this. Minsky & Papert, 1969.



The fix: add a hidden layer

- $XOR = (x_1 \text{ OR } x_2) \text{ AND NOT}(x_1 \text{ AND } x_2)$
- A hidden neuron computes OR, another computes NAND.
- The output neuron ANDs those two hidden results.
- The hidden layer re-represents the input in a space where the classes ARE linearly separable.

A multi-layer feedforward network (MLP)



Key terms

Fully connected: every node links to every node in the next layer.

Depth = number of layers.

Width = nodes per layer.

Feedforward = no cycles.

Notation we will use throughout

$\mathbf{x}$	input vector	shape $(n_0 \times 1)$	$\hat{\mathbf{y}}$	network prediction	$\hat{\mathbf{y}} = \mathbf{a}^{(z)}$, last layer
$\mathbf{W}^{(\ell)}$	weight matrix of layer ℓ	shape $(n_\ell \times n_{\ell-1})$	$\mathbf{y}$	true target / gold label	from the training data
$w^{(\ell)}_{jk}$	weight from node k in layer $\ell-1$ to node j in layer ℓ	scalar	$\mathbf{L}$	loss for one example	scalar
$\mathbf{b}^{(\ell)}$	bias vector of layer ℓ	shape $(n_\ell \times 1)$	$\delta^{(\ell)}$	error signal of layer ℓ	$\delta^{(\ell)} = \partial L / \partial \mathbf{z}^{(\ell)}$
$\mathbf{z}^{(\ell)}$	pre-activation (net input) of layer ℓ	$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$	η	learning rate	small positive scalar
$\mathbf{a}^{(\ell)}$	activation (output) of layer ℓ	$\mathbf{a}^{(\ell)} = \varphi(\mathbf{z}^{(\ell)})$, $\mathbf{a}^{(0)} = \mathbf{x}$			

Superscripts in brackets index LAYERS, not powers. Subscripts index NODES. Read $w^{(2)}_{31}$ as: layer 2, into node 3, from node 1.

Activation Functions

The non-linearity that gives a network its power. Without it, a hundred layers collapse into one.

- Why non-linearity is mathematically necessary
- Step, Sigmoid, Tanh, ReLU, Leaky ReLU, ELU, GELU, Swish
- Softmax for multi-class output, with the stability trick
- Derivatives, value tables, and how to choose

Why a network without activations is useless

Proof: stacking linear layers gains nothing

$$\text{Layer 1: } a^{(1)} = W^{(1)}x + b^{(1)}$$

$$\text{Layer 2: } a^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$

Substitute:

$$a^{(2)} = W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)}$$

$$a^{(2)} = (W^{(2)}W^{(1)})x + (W^{(2)}b^{(1)} + b^{(2)})$$

$$a^{(2)} = W'x + b'$$

Two layers reduced to one. Ten layers reduce to one. The model can only ever draw a straight line.

With a non-linearity ϕ

$$a^{(1)} = \phi(W^{(1)}x + b^{(1)})$$

$$a^{(2)} = \phi(W^{(2)}a^{(1)} + b^{(2)})$$

ϕ cannot be factored out.

No collapse is possible.

Universal Approximation Theorem (Cybenko 1989; Hornik 1991): a feedforward network with one hidden layer and a non-constant, bounded, continuous activation can approximate any continuous function on a compact domain to arbitrary accuracy — given enough hidden units.

Three jobs an activation function does: (1) introduce non-linearity (2) bound or shape the output range (3) supply a usable gradient for learning

Step function and linear (identity)

Binary step (Heaviside)

$$\varphi(z) = 1 \text{ if } z \geq 0$$

$$\varphi(z) = 0 \text{ if } z < 0$$

$$\varphi'(z) = 0 \text{ everywhere it exists}$$

undefined at $z = 0$

Verdict

- Used by the original 1958 perceptron.
- Gradient is zero, so gradient descent learns nothing.
- Not usable in any network trained by backpropagation.
- Historical interest only — but it explains why we needed sigmoid.

Linear / identity

$$\varphi(z) = z$$

$$\varphi'(z) = 1 \text{ for all } z$$

Verdict

- Constant gradient — no vanishing, but also no non-linearity.
- Stacking linear layers collapses (previous slide).
- Legitimate in ONE place: the output layer of a regression model, where the prediction must be unbounded.
- In NLP: predicting a continuous score, e.g. essay grading.

Sigmoid (logistic) — the classic squashing function

$$\sigma(z) = 1 / (1 + e^{-z}) \quad \sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

Range & shape

output $\in (0, 1)$
 $\sigma(0) = 0.5$
S-shaped, monotonic
not zero-centred

Derivative

$\max \sigma'(0) = 0.25$
 $\sigma'(\pm 5) \approx 0.0066$
→ saturates fast
expressed via σ itself

Where to use it

binary classification
output layer only
LSTM / GRU gates
not hidden layers

Advantages

- Smooth and differentiable everywhere.
- Output reads directly as a probability.
- Derivative is cheap: reuse the forward output.

Problems

- Vanishing gradients: $\sigma' \leq 0.25$ kills deep nets.
- Not zero-centred → zig-zag gradient updates.
- $\exp()$ is expensive compared with ReLU.

Tanh — the zero-centred sigmoid

$$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z}) \quad \tanh'(z) = 1 - \tanh^2(z)$$

Relationship to sigmoid

$$\tanh(z) = 2 \cdot \sigma(2z) - 1$$

It is a rescaled, re-centred sigmoid:
range $(-1, 1)$ instead of $(0, 1)$.

Why zero-centred matters

With sigmoid, every activation is positive, so all gradients feeding one neuron share the same sign — weights are forced to move together, producing slow zig-zag descent. Tanh outputs both signs and removes this bias.

Advantages

- Zero-centred output
- Steeper gradient than sigmoid (max 1.0 vs 0.25)
- Strong for shallow nets and RNN hidden states

Problems

- Still saturates at both ends
- Still vanishing gradients when $|z|$ is large
- Two exponentials to compute

NLP usage

- Hidden state of vanilla RNNs
- LSTM cell candidate values
- Older word-embedding models

ReLU — the default choice for hidden layers

$$\text{ReLU}(z) = \max(0, z) \quad \text{ReLU}'(z) = 1 \text{ if } z > 0, \text{ else } 0$$

Why it won

- Gradient is exactly 1 for positive z — no shrinkage, so signals survive 50+ layers.
- A comparison and a max: far cheaper than $\exp()$.
- Sparse activation — roughly half the units output 0, which acts as built-in regularisation.
- Empirically converges several times faster than sigmoid or tanh.

The dying ReLU problem

If a large gradient update pushes the weights so that $z < 0$ for every training example, the unit outputs 0 and its gradient is 0. No gradient reaches it, so it can never recover. The neuron is permanently dead.

Mitigations: lower the learning rate, use He initialisation, or switch to Leaky ReLU / ELU.

You will see this happen in Example 2

Later in this lecture, hidden unit h_2 receives $z = -0.25$. ReLU outputs 0, its derivative is 0, and every gradient flowing into w_{21} , w_{22} and b_2 comes out as exactly 0.000000 — those three parameters do not move at all during that update. Watch for it.

Repairing ReLU: Leaky ReLU, PReLU, ELU

Leaky ReLU

$$\begin{aligned}\varphi(z) &= z & \text{if } z > 0 \\ \varphi(z) &= \alpha z & \text{if } z \leq 0\end{aligned}$$

α fixed, typically 0.01

Gives dead units a small escape gradient of α instead of 0. Cheap, and almost always at least as good as plain ReLU.

Use when: ReLU units are dying and you want a one-line fix.

PReLU

$$\begin{aligned}\varphi(z) &= z & \text{if } z > 0 \\ \varphi(z) &= \alpha z & \text{if } z \leq 0\end{aligned}$$

α is LEARNED per channel

The slope becomes a trainable parameter. Slightly better accuracy at the cost of extra parameters and mild overfitting risk.

Use when: you have plenty of data and want the last point of accuracy.

ELU

$$\begin{aligned}\varphi(z) &= z & \text{if } z > 0 \\ \varphi(z) &= \alpha(e^z - 1) & \text{if } z \leq 0\end{aligned}$$

α typically 1.0

Smooth everywhere and pushes the mean activation toward zero, which speeds up learning. But $\exp()$ makes it slower to compute.

Use when: accuracy matters more than training speed.

At $z = -2$ with $\alpha = 0.01$: ReLU $\rightarrow 0.00$ · Leaky ReLU $\rightarrow -0.02$ · ELU ($\alpha = 1$) $\rightarrow -0.8647$

Swish and GELU — what modern NLP models actually use

Swish / SiLU

$$\text{swish}(z) = z \cdot \sigma(z)$$

- Smooth, non-monotonic: dips slightly below zero near $z \approx -1.3$.
- Self-gating — the input scales itself.
- $\text{swish}(1) = 1 \times 0.7311 = 0.7311$; $\text{swish}(-2) = -2 \times 0.1192 = -0.2384$

GELU (Gaussian Error Linear Unit)

$$\text{GELU}(z) = z \cdot \Phi(z)$$

Φ = standard normal CDF

- Weights each input by the probability that a standard normal is below it — a smooth, probabilistic gate.
- $\text{GELU}(1) \approx 0.8413$; $\text{GELU}(-2) \approx -0.0455$

Which activation does each famous NLP model use?

BERT

GELU

GPT-2 / GPT-3

GELU

T5

ReLU

LLaMA / PaLM

SwiGLU

Vanilla RNN

tanh

LSTM gates

sigmoid

Takeaway for exams: ReLU is the safe default for hidden layers; GELU is the transformer standard; sigmoid and softmax belong in the output layer.

Softmax — turning scores into a probability distribution

$$\text{softmax}(z)_i = e^{(z_i)} / \sum_j e^{(z_j)}$$

- Every output is in (0,1) and all outputs sum to exactly 1.
- Used at the output layer for multi-class problems — in NLP: POS tagging, language modelling over a vocabulary.

Numerical stability — always do this

`e^1000` overflows to infinity.

Subtract the max first — the result is mathematically identical:

`softmax(z) = softmax(z - max(z))`

Worked example — 3-class POS tagger

Logits	<code>z = [2.0 , 1.0 , 0.1]</code>
Classes	NOUN VERB ADJ

`e^2.0 = 7.3891`

`e^1.0 = 2.7183`

`e^0.1 = 1.1052`

`sum = 11.2126`

`P(NOUN) = 7.3891 / 11.2126 = 0.6590`

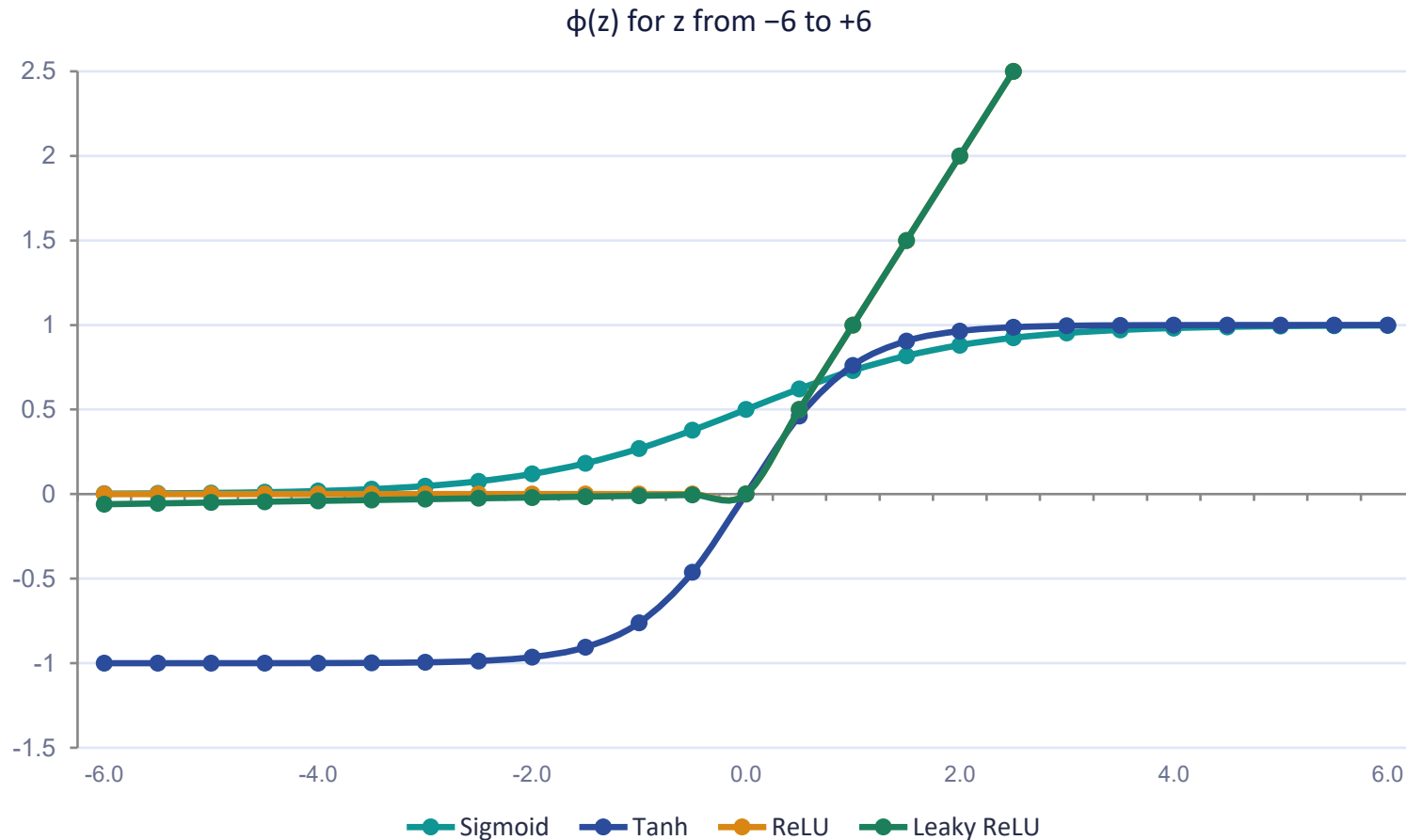
`P(VERB) = 2.7183 / 11.2126 = 0.2424`

`P(ADJ) = 1.1052 / 11.2126 = 0.0986`

`total = 1.0000 ✓`

Notice: a logit gap of 1.0 became a probability ratio of 2.7×. Softmax amplifies differences exponentially.

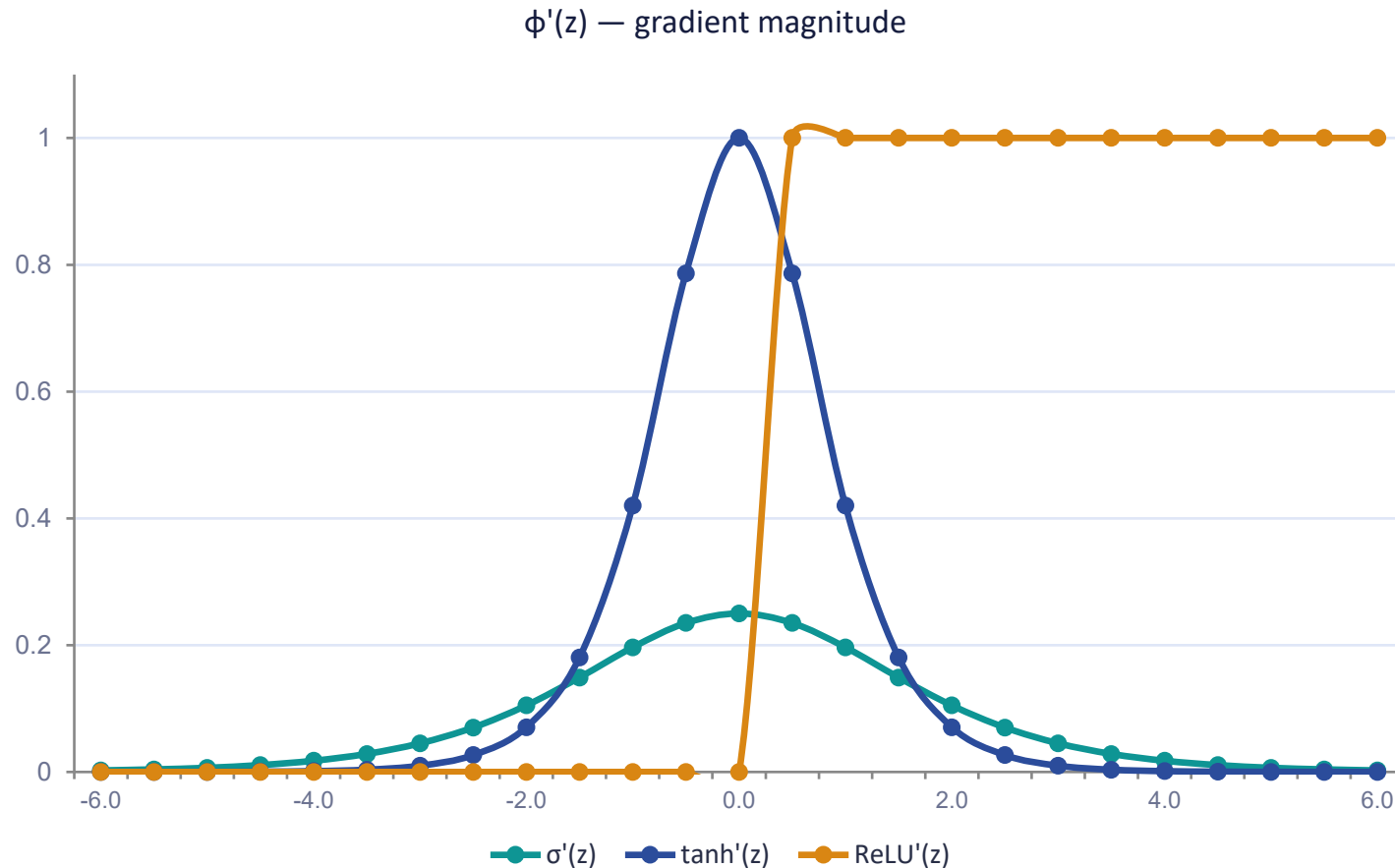
The four main activation curves



What to notice

- Sigmoid and tanh both flatten out — the flat regions are where gradients die.
- Sigmoid never goes negative; tanh is symmetric about the origin.
- ReLU is unbounded above — it is clipped at 2.5 here, but it keeps rising forever.
- Leaky ReLU is visually almost identical to ReLU — the $\alpha = 0.01$ slope is deliberately tiny.
- All four are monotonic; only Swish and GELU are not.

Derivatives decide whether a network can train



The killer detail

$$\max \sigma' = 0.25$$

$$\max \tanh' = 1.00$$

$$\text{ReLU}' = 1 \text{ (for } z > 0\text{)}$$

Sigmoid's gradient is capped at one quarter. Every layer you add multiplies the signal by at most 0.25.

Derivative formulas

$$\sigma'(z) = \sigma(1-\sigma)$$

$$\tanh'(z) = 1 - \tanh^2$$

$$\text{ReLU}'(z) = 1 \text{ if } z > 0 \text{ else } 0$$

$$\text{Leaky}' = 1 \text{ if } z > 0 \text{ else } \alpha$$

$$\text{ELU}' = 1 \text{ if } z > 0 \\ \text{else } \phi(z) + \alpha$$

Activation value table — check your own arithmetic

z	$\sigma(z)$	$\sigma'(z)$	$\tanh(z)$	$\tanh'(z)$	$\text{ReLU}(z)$	$\text{ReLU}'(z)$
-3.0	0.0474	0.0452	-0.9951	0.0099	0.00	0
-2.0	0.1192	0.1050	-0.9640	0.0707	0.00	0
-1.0	0.2689	0.1966	-0.7616	0.4200	0.00	0
-0.5	0.3775	0.2350	-0.4621	0.7864	0.00	0
0.0	0.5000	0.2500	0.0000	1.0000	0.00	0* ◀ peak gradient
0.5	0.6225	0.2350	0.4621	0.7864	0.50	1
1.0	0.7311	0.1966	0.7616	0.4200	1.00	1
2.0	0.8808	0.1050	0.9640	0.0707	2.00	1
3.0	0.9526	0.0452	0.9951	0.0099	3.00	1

* ReLU is not differentiable at exactly $z = 0$. Every library simply defines the derivative there as 0 (PyTorch, TensorFlow) — it never matters in practice, since hitting exactly 0.0 has probability zero.

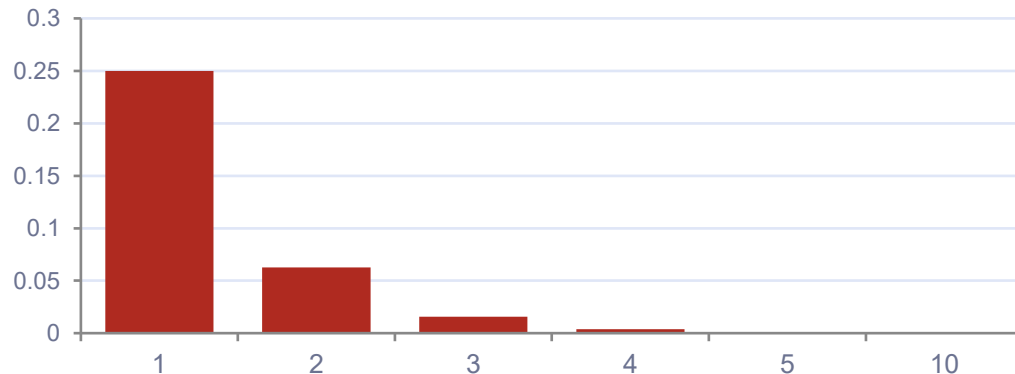
Vanishing gradients, quantified

Why depth breaks sigmoid networks

Backpropagation multiplies one $\phi'(z)$ factor per layer. With sigmoid, each factor is at most 0.25, so the gradient reaching layer 1 of an n -layer network is scaled by at most 0.25^n .

$$\partial L / \partial \mathbf{w}^{(1)} \leq (0.25)^n \times (\text{weight terms})$$

$(0.25)^n$ by number of sigmoid layers (log scale)



The numbers

1 layer	→	0.25	(25%)
2 layers	→	0.0625	(6.3%)
3 layers	→	0.015625	(1.6%)
5 layers	→	0.00097656	(0.1%)
10 layers	→	0.00000095	(0.0001%)

By layer 10 the gradient is a millionth of its original size. The early layers — exactly the ones that learn word features in NLP — stop learning entirely.

How the field fixed it

- ReLU family — gradient of exactly 1 on the positive side
- Residual (skip) connections — a gradient highway around layers
- Batch / layer normalisation — keeps z near the high-gradient region
- Careful initialisation — Xavier for tanh, He for ReLU

Which activation should you use?

● Hidden layers, general	ReLU	Fast, simple, no vanishing gradient. Start here.
● Hidden layers, if units die	Leaky ReLU or ELU	Keeps a small gradient alive on the negative side.
● Hidden layers, transformers	GELU or SwiGLU	Smoother gating; the empirical standard for BERT/GPT.
● RNN / LSTM hidden state	tanh	Zero-centred and bounded, which keeps recurrence stable.
● LSTM / GRU gates	sigmoid	Output in (0,1) reads directly as “how much to let through”.
● Output — binary class	sigmoid	One probability. Pair with binary cross-entropy.
● Output — multi-class	softmax	A distribution over classes. Pair with cross-entropy.
● Output — multi-label	sigmoid per label	Labels are independent, so they need not sum to 1.
● Output — regression	linear (none)	The prediction must be free to take any real value.

Rule of thumb: ReLU in the middle, and let the task decide the end.

Feedforward Propagation

Push the input through the network, one layer at a time, and read off a prediction.

- The layer-by-layer scalar computation
- The same thing in matrix form — what every library actually runs
- Loss functions: MSE, binary cross-entropy, categorical cross-entropy
- Example 1: a complete forward pass computed by hand

Feedforward: two operations, repeated per layer

For each layer $\ell = 1, 2, \dots, L$

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)} \quad (\text{linear step})$$

$$a^{(\ell)} = \varphi^{(\ell)}(z^{(\ell)}) \quad (\text{non-linear step})$$

with $a^{(0)} = x$ (the input) and $\hat{y} = a^{(L)}$ (the prediction)

Scalar form

$$z_j^{(\ell)} = \sum_k w_{jk}^{(\ell)} a_k^{(\ell-1)} + b_j^{(\ell)}$$

$$a_j^{(\ell)} = \varphi(z_j^{(\ell)})$$

j indexes the neuron

k indexes its inputs

1 Place the input

Set $a^{(0)} = x$. No computation happens in the input layer — it only holds values.

2 Linear combination

Multiply by the weight matrix and add the bias: $z = Wa + b$. This is a rotation and a shift.

3 Apply activation

Squash element-wise: $a = \phi(z)$. This is where the model gains its expressive power.

4 Repeat to the end

Feed a forward as the next layer's input. Do this L times, then read $\hat{y}$ from the final layer.

Matrix form — what the GPU actually computes

One example: shapes must line up

$$\begin{array}{ccccccc} z^{(1)} & = & W^{(1)} & \times & a^{(1-1)} & + & b^{(1)} \\ (n_l \times 1) & & (n_l \times n_{l-1}) & & (n_{l-1} \times 1) & & (n_l \times 1) \end{array}$$

Example: 300-dim embedding $\rightarrow$ 128 hidden units

$$\begin{array}{ccccccc} z^{(1)} & = & W^{(1)} & \times & x & + & b^{(1)} \\ (128 \times 1) & & (128 \times 300) & & (300 \times 1) & & (128 \times 1) \end{array}$$

Mini-batch of m examples

$$Z^{(l)} = W^{(l)} A^{(l-1)} + b^{(l)}$$

$A^{(l-1)}$ is $(n_{l-1} \times m)$:

one column per example.

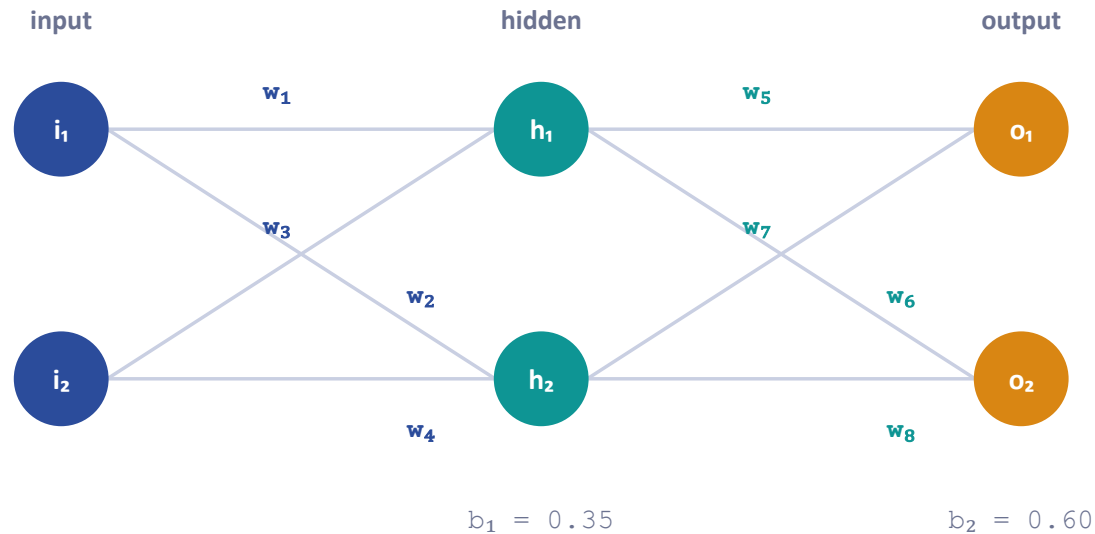
The bias is broadcast
across all m columns.

One matrix multiply now
handles the whole batch.

A concrete NLP shape walkthrough — sentiment classifier over a 300-d sentence embedding

$$\begin{array}{l} x \quad (300 \times 1) \quad \rightarrow \quad W^{(1)} \quad (128 \times 300) \quad \times \quad x \quad + \quad b^{(1)} \quad (128 \times 1) \quad \rightarrow \quad z^{(1)} \quad (128 \times 1) \quad \rightarrow \quad a^{(1)} = \text{ReLU}(z^{(1)}) \quad (128 \times 1) \\ a^{(1)} \quad (128 \times 1) \quad \rightarrow \quad W^{(2)} \quad (1 \times 128) \quad \times \quad a^{(1)} \quad + \quad b^{(2)} \quad (1 \times 1) \quad \rightarrow \quad z^{(2)} \quad (1 \times 1) \quad \rightarrow \quad \hat{y} = \sigma(z^{(2)}) \quad (1 \times 1) \end{array}$$

A 2-2-2 network we will train by hand



Weight naming convention on this diagram

$w_1: i_1 \rightarrow h_1$ $w_2: i_2 \rightarrow h_1$ $w_3: i_1 \rightarrow h_2$ $w_4: i_2 \rightarrow h_2$
 $w_5: h_1 \rightarrow o_1$ $w_6: h_2 \rightarrow o_1$ $w_7: h_1 \rightarrow o_2$ $w_8: h_2 \rightarrow o_2$

Given values

Inputs $i_1 = 0.05$ $i_2 = 0.10$
 Targets $t_1 = 0.01$ $t_2 = 0.99$

$w_1 = 0.15$ $w_2 = 0.20$ $b_1 = 0.35$
 $w_3 = 0.25$ $w_4 = 0.30$
 $w_5 = 0.40$ $w_6 = 0.45$ $b_2 = 0.60$
 $w_7 = 0.50$ $w_8 = 0.55$

Design choices

Activation : sigmoid (both layers)

Loss : $E = \frac{1}{2} \sum (\text{target} - \text{out})^2$

Learning η : 0.5

The $\frac{1}{2}$ is there purely so the 2 cancels when we differentiate.

Class Activity = 02

Randomly initialize the Weights to small number close to 0 (but not 0)

Weights for Layer 1

$$W_{ij} = \begin{bmatrix} W_{i1j1} & W_{i1j2} & W_{i1j3} \\ W_{i2j1} & W_{i2j2} & W_{i2j3} \\ W_{i3j1} & W_{i3j2} & W_{i3j3} \end{bmatrix} \quad W_{ij} = \begin{bmatrix} 0.2 & 0.4 & 0.5 \\ 0.3 & 0.7 & 0.1 \\ 0.2 & 0.6 & 0.9 \end{bmatrix}$$

Inputs

$$\text{Inputs} = [i_1 \quad i_2 \quad i_3]$$

$$= [0.1 \quad 0.3 \quad 0.5]$$

Weights for Layer 2

$$W_{jk} = \begin{bmatrix} W_{j1k1} & W_{j1k2} & W_{j1k3} \\ W_{j2k1} & W_{j2k2} & W_{j2k3} \\ W_{j3k1} & W_{j3k2} & W_{j3k3} \end{bmatrix} \quad W_{jk} = \begin{bmatrix} 0.4 & 0.6 & 0.1 \\ 0.2 & 0.7 & 0.3 \\ 0.8 & 0.1 & 0.5 \end{bmatrix}$$

Actual Outputs

$$\text{Outputs} = [y_1 \quad y_2 \quad y_3]$$

$$= [0.0 \quad 1.0 \quad 0.0]$$

Weights for Layer 3

$$W_{kl} = \begin{bmatrix} W_{k1l1} & W_{k1l2} & W_{k1l3} \\ W_{k2l1} & W_{k2l2} & W_{k2l3} \\ W_{k3l1} & W_{k3l2} & W_{k3l3} \end{bmatrix} \quad W_{kl} = \begin{bmatrix} 0.2 & 0.5 & 0.3 \\ 0.4 & 0.6 & 0.8 \\ 0.1 & 0.9 & 0.7 \end{bmatrix}$$

Hidden layer: compute net input, then activate

Neuron h_1

$$\begin{aligned}\text{net_}h_1 &= w_1 \cdot i_1 + w_2 \cdot i_2 + b_1 \\ &= (0.15)(0.05) + (0.20)(0.10) + 0.35 \\ &= 0.0075 + 0.0200 + 0.35 \\ \text{net_}h_1 &= 0.3775\end{aligned}$$

$$\text{out_}h_1 = 1 / (1 + e^{(-0.3775)}) = 0.593270$$

Neuron h_2

$$\begin{aligned}\text{net_}h_2 &= w_3 \cdot i_1 + w_4 \cdot i_2 + b_1 \\ &= (0.25)(0.05) + (0.30)(0.10) + 0.35 \\ &= 0.0125 + 0.0300 + 0.35 \\ \text{net_}h_2 &= 0.3925\end{aligned}$$

$$\text{out_}h_2 = 1 / (1 + e^{(-0.3925)}) = 0.596884$$

Same computation in matrix form — verify you get the identical answer

$$z^{(1)} = [0.15 \quad 0.20 ; 0.25 \quad 0.30] \times [0.05 ; 0.10] + [0.35 ; 0.35] = [0.3775 ; 0.3925] \quad a^{(1)} = \sigma(z^{(1)}) = [0.593270 ; 0.596884]$$

Sanity check: both net inputs are positive but small, so both sigmoid outputs land just above 0.5. Since h_1 and h_2 share the same bias and see the same inputs, their outputs differ only because their weights differ — and only slightly.

Output layer: the hidden activations become the new inputs

Neuron o_1

$$\begin{aligned} \text{net}_{o_1} &= w_5 \cdot \text{out}_{h_1} + w_6 \cdot \text{out}_{h_2} + b_2 \\ &= (0.40)(0.593270) \\ &\quad + (0.45)(0.596884) + 0.60 \\ &= 0.237308 + 0.268598 + 0.60 \\ \text{net}_{o_1} &= 1.105906 \end{aligned}$$

$$\text{out}_{o_1} = \sigma(1.105906) = 0.751365$$

Neuron o_2

$$\begin{aligned} \text{net}_{o_2} &= w_7 \cdot \text{out}_{h_1} + w_8 \cdot \text{out}_{h_2} + b_2 \\ &= (0.50)(0.593270) \\ &\quad + (0.55)(0.596884) + 0.60 \\ &= 0.296635 + 0.328286 + 0.60 \\ \text{net}_{o_2} &= 1.224921 \end{aligned}$$

$$\text{out}_{o_2} = \sigma(1.224921) = 0.772928$$

Where we stand after the forward pass

	predicted	target	error
o_1	0.751365	0.010000	far too high
o_2	0.772928	0.990000	too low

The network is badly wrong — it has never been trained. Backpropagation is how we fix it.

Loss functions — how wrong is the prediction?

Mean Squared Error

$$E = \frac{1}{2} \sum (t_i - \hat{y}_i)^2$$

gradient at the output:

$$\partial E / \partial \hat{y} = \hat{y} - t$$

Regression. Used in Example 1 because it makes the hand arithmetic transparent.

Binary Cross-Entropy

$$L = -[y \cdot \ln \hat{y} + (1-y) \cdot \ln(1-\hat{y})]$$

gradient at the output:

$$\partial L / \partial z = \hat{y} - y$$

Binary classification with a sigmoid output. Used in Example 2.

Categorical Cross-Entropy

$$L = - \sum_i y_i \cdot \ln \hat{y}_i$$

gradient at the output:

$$\partial L / \partial z = \hat{y} - y$$

Multi-class with a softmax output. Used in Example 3. In NLP: tagging, language modelling.

Why cross-entropy dominates classification: pairing it with sigmoid or softmax makes the output gradient collapse to the beautifully simple $\hat{y} - y$, and it does not suffer the gradient slowdown that MSE + sigmoid produces when the model is confidently wrong.

Computing the total error

Error at o_1

$$\begin{aligned}
 E_{o_1} &= \frac{1}{2} (t_1 - \text{out}_{o_1})^2 \\
 &= \frac{1}{2} (0.010000 - 0.751365)^2 \\
 &= \frac{1}{2} (-0.741365)^2 \\
 &= \frac{1}{2} (0.549622) \\
 E_{o_1} &= 0.274811
 \end{aligned}$$

Error at o_2

$$\begin{aligned}
 E_{o_2} &= \frac{1}{2} (t_2 - \text{out}_{o_2})^2 \\
 &= \frac{1}{2} (0.990000 - 0.772928)^2 \\
 &= \frac{1}{2} (0.217072)^2 \\
 &= \frac{1}{2} (0.047120) \\
 E_{o_2} &= 0.023560
 \end{aligned}$$

$$E_{\text{total}} = E_{o_1} + E_{o_2} = 0.274811 + 0.023560 = 0.298371$$

Almost all the error comes from o_1 — 92% of it. Expect the gradients feeding o_1 to be much larger than those feeding o_2 .

Forward pass complete. We now have a number that measures failure. Everything that follows answers one question: how should each of the ten parameters change to make 0.298371 smaller?

PART 04

Backpropagation

Work backwards from the error and ask every weight: how much of this mistake was your fault?

- The chain rule, and why it is the whole algorithm
- The four backpropagation equations
- Example 1 continued: every gradient computed by hand
- Updating all ten parameters, then proving the loss went down



04

Credit assignment by the chain rule

The problem

We have one number, $E = 0.298371$. We have ten parameters. Each parameter influenced E through a different path — sometimes several paths at once. We need $\partial E / \partial w$ for every single one, and we need it cheaply.

Naive approach: nudge each weight and re-run the network. For 175 billion parameters that is 175 billion forward passes per step. Hopeless.

The solution

Backpropagation computes all gradients in a single backward sweep, at roughly the same cost as one forward pass — regardless of how many parameters there are.

It does this by reusing partial results: the error signal computed at layer ℓ is exactly what layer $\ell-1$ needs. Nothing is recomputed.

Chain rule refresher

If $y = f(u)$ and $u = g(x)$, then $dy/dx = (dy/du) \times (du/dx)$. Chained through a network: $\partial E / \partial w = (\partial E / \partial \text{out}) \times (\partial \text{out} / \partial \text{net}) \times (\partial \text{net} / \partial w)$

$\partial E / \partial \text{out}$

How much does the error change if this neuron's output changes?

Comes from the loss function (output layer) or from the layer above (hidden layers).

$\partial \text{out} / \partial \text{net}$

How much does the output change if the net input changes?

This is exactly $\phi'(\text{net})$ — the activation derivative. This is why activation choice matters.

$\partial \text{net} / \partial w$

How much does the net input change if this weight changes?

Just the incoming activation. $\partial \text{net} / \partial w_{jk} = a_k$. Nothing to compute — you already have it.

The four equations of backpropagation

BP1

Error at the output layer

$$\delta^{(z)} = \nabla_a L \odot \varphi'(z^{(z)})$$

How wrong the prediction is, scaled by how responsive the output neuron is. With sigmoid + MSE: $\delta = (\hat{y} - t) \cdot \hat{y}(1 - \hat{y})$. With sigmoid + BCE, or softmax + CE: $\delta = \hat{y} - y$.

BP2

Error propagated backwards

$$\delta^{(l)} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot \varphi'(z^{(l)})$$

Push the error one layer back through the transposed weights, then gate it by this layer's activation derivative. This recursion is the heart of the algorithm.

BP3

Gradient for any bias

$$\partial L / \partial b^{(l)}_j = \delta^{(l)}_j$$

The bias gradient IS the error signal — because $\partial z / \partial b = 1$.

BP4

Gradient for any weight

$$\partial L / \partial w^{(l)}_{jk} = a^{(l-1)}_k \cdot \delta^{(l)}_j$$

Error signal at the destination $\times$ activation at the source. Note: if the input a_k is 0, the gradient is 0 — that weight does not move.

$\odot$ means element-wise (Hadamard) product. Then update every parameter: $w \leftarrow w - \eta \cdot \partial L / \partial w$

Deriving $\partial E / \partial w_5$ from scratch

Split it with the chain rule

$$\frac{\partial E}{\partial w_5} = \frac{\partial E}{\partial \text{out}_{o_1}} \times \frac{\partial \text{out}_{o_1}}{\partial \text{net}_{o_1}} \times \frac{\partial \text{net}_{o_1}}{\partial w_5}$$

Factor 1 — differentiate the loss

$$E = \frac{1}{2}(t_1 - \text{out}_{o_1})^2 + \frac{1}{2}(t_2 - \text{out}_{o_2})^2$$

$$\partial E / \partial \text{out}_{o_1} = -(t_1 - \text{out}_{o_1}) = \text{out}_{o_1} - t_1$$

Factor 2 — differentiate the sigmoid

$$\text{out}_{o_1} = \sigma(\text{net}_{o_1})$$

$$\partial \text{out}_{o_1} / \partial \text{net}_{o_1} = \text{out}_{o_1}(1 - \text{out}_{o_1})$$

Factor 3 — differentiate the linear step

$$\text{net}_{o_1} = w_5 \cdot \text{out}_{h_1} + w_6 \cdot \text{out}_{h_2} + b_2$$

$$\partial \text{net}_{o_1} / \partial w_5 = \text{out}_{h_1}$$

Put the three together

$$\begin{aligned} \partial E / \partial w_5 = & \\ & (\text{out}_{o_1} - t_1) \\ & \times \text{out}_{o_1}(1 - \text{out}_{o_1}) \\ & \times \text{out}_{h_1} \end{aligned}$$

The first two factors depend only on neuron o_1 . Name them δ_{o_1} and reuse for w_6 and b_2 .

This is where δ comes from

$$\delta_{o_1} = (\text{out}_{o_1} - t_1) \cdot \sigma'(\text{net}_{o_1})$$

$$\begin{aligned} \partial E / \partial w_5 &= \delta_{o_1} \cdot \text{out}_{h_1} \\ \partial E / \partial w_6 &= \delta_{o_1} \cdot \text{out}_{h_2} \\ \partial E / \partial b_2 &= \delta_{o_1} + \delta_{o_2} \end{aligned}$$

Error signals at the output layer

 δ_{o_1}

$$\begin{aligned}\text{out}_{o_1} - t_1 &= 0.751365 - 0.010000 \\ &= 0.741365\end{aligned}$$

$$\begin{aligned}\sigma'(\text{net}_{o_1}) &= 0.751365 \times (1 - 0.751365) \\ &= 0.751365 \times 0.248635 = 0.186816\end{aligned}$$

$$\delta_{o_1} = 0.741365 \times 0.186816 = 0.138499$$

 δ_{o_2}

$$\begin{aligned}\text{out}_{o_2} - t_2 &= 0.772928 - 0.990000 \\ &= -0.217072\end{aligned}$$

$$\begin{aligned}\sigma'(\text{net}_{o_2}) &= 0.772928 \times (1 - 0.772928) \\ &= 0.772928 \times 0.227072 = 0.175510\end{aligned}$$

$$\delta_{o_2} = -0.217072 \times 0.175510 = -0.038098$$

Read the signs — they tell you which way each weight will move

δ_{o_1} is positive: o_1 predicted too high, so the weights feeding it must decrease. δ_{o_2} is negative: o_2 predicted too low, so the weights feeding it must increase.

$|\delta_{o_1}| = 0.138499$ is 3.6× larger than $|\delta_{o_2}| = 0.038098$ — exactly as predicted from the error split (92% vs 8%). Bigger mistakes produce bigger corrections.

Gradients and updates for $w_5 - w_8$

gradient	formula	substitution	value	old w	new w = old - 0.5·grad
$\partial E / \partial w_5$	$\delta_{o_1} \times \text{out}_{h_1}$	0.138499×0.593270	0.082167	0.40	0.358916
$\partial E / \partial w_6$	$\delta_{o_1} \times \text{out}_{h_2}$	0.138499×0.596884	0.082668	0.45	0.408666
$\partial E / \partial w_7$	$\delta_{o_2} \times \text{out}_{h_1}$	$-0.038098 \times 0.593270$	-0.022603	0.50	0.511301
$\partial E / \partial w_8$	$\delta_{o_2} \times \text{out}_{h_2}$	$-0.038098 \times 0.596884$	-0.022740	0.55	0.561370

Bias b_2

$$\begin{aligned}
 \partial E / \partial b_2 &= \delta_{o_1} + \delta_{o_2} \\
 &= 0.138499 + (-0.038098) = 0.100400 \\
 b_2^+ &= 0.60 - 0.5(0.100400) = 0.549800
 \end{aligned}$$

Check the direction

w_5 and w_6 went DOWN (they fed the over-confident o_1). w_7 and w_8 went UP (they fed the under-confident o_2). The gradient found the right direction without being told.

Hidden deltas — the step students get wrong

Why this step is different

out_h1 was sent to BOTH output neurons. It therefore influenced the error along two separate paths, so its error contributions must be SUMMED. Forgetting the second path is the single most common mistake in backpropagation exercises.

δ_{h_1}

$$\begin{aligned}\partial E / \partial \text{out}_{h_1} &= \delta_{o_1} \cdot w_5 + \delta_{o_2} \cdot w_7 \\ &= (0.138499)(0.40) + (-0.038098)(0.50) \\ &= 0.055399 + (-0.019049) \\ &= 0.036350\end{aligned}$$

$$\sigma'(\text{net}_{h_1}) = 0.593270 \times 0.406730 = 0.241301$$

$$\delta_{h_1} = 0.036350 \times 0.241301 = 0.008771$$

δ_{h_2}

$$\begin{aligned}\partial E / \partial \text{out}_{h_2} &= \delta_{o_1} \cdot w_6 + \delta_{o_2} \cdot w_8 \\ &= (0.138499)(0.45) + (-0.038098)(0.55) \\ &= 0.062324 + (-0.020954) \\ &= 0.041370\end{aligned}$$

$$\sigma'(\text{net}_{h_2}) = 0.596884 \times 0.403116 = 0.240613$$

$$\delta_{h_2} = 0.041370 \times 0.240613 = 0.009954$$

Note we use the OLD values of w_5 – w_8 , not the updated ones. All gradients are computed with respect to the same parameter state, then everything is updated at once.

Gradients and updates for $w_1 - w_4$

gradient	formula	substitution	value	old w	new w = old - 0.5·grad
$\partial E / \partial w_1$	$\delta_{h_1} \times i_1$	0.008771×0.05	0.000439	0.15	0.149781
$\partial E / \partial w_2$	$\delta_{h_1} \times i_2$	0.008771×0.10	0.000877	0.20	0.199561
$\partial E / \partial w_3$	$\delta_{h_2} \times i_1$	0.009954×0.05	0.000498	0.25	0.249751
$\partial E / \partial w_4$	$\delta_{h_2} \times i_2$	0.009954×0.10	0.000995	0.30	0.299502

Bias b_1

$$\begin{aligned}
 \partial E / \partial b_1 &= \delta_{h_1} + \delta_{h_2} \\
 &= 0.008771 + 0.009954 = 0.018726 \\
 b_1^+ &= 0.35 - 0.5(0.018726) = 0.340637
 \end{aligned}$$

Why are these gradients so tiny?

The inputs are 0.05 and 0.10 — very small — and the sigmoid derivative shrank the signal twice on the way back. This is the vanishing gradient, visible in a two-layer toy network.

All ten parameters after one training step

parameter	before	gradient	after	parameter	before	gradient	after
w_1	0.150000	0.000439	0.149781	w_5	0.400000	0.082167	0.358916
w_2	0.200000	0.000877	0.199561	w_6	0.450000	0.082668	0.408666
w_3	0.250000	0.000498	0.249751	w_7	0.500000	-0.022603	0.511301
w_4	0.300000	0.000995	0.299502	w_8	0.550000	-0.022740	0.561370
b_1	0.350000	0.018726	0.340637	b_2	0.600000	0.100400	0.549800

Notice the asymmetry

Output-layer weights moved by up to 0.041; hidden-layer weights moved by less than 0.0005 — nearly a hundred times less. Layers close to the loss learn fastest; layers far from it learn slowly. That gap is the central difficulty of training deep networks.

Did the error actually go down?

Re-run the forward pass with the new parameters

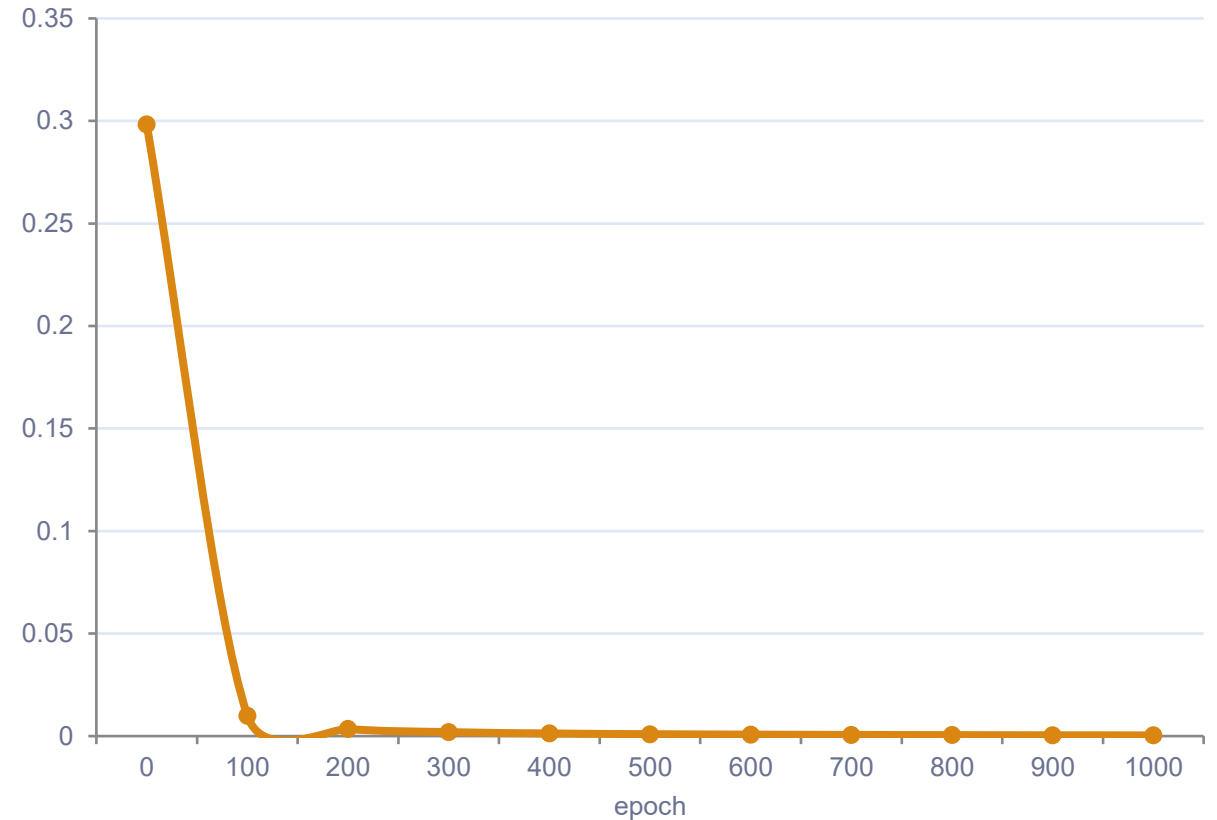
	before	after
out_o ₁	0.751365	0.732024
out_o ₂	0.772928	0.765985
E _{total}	0.298371	0.285751
$\Delta E = -0.012620$ (-4.2% in one step)		

After repeating this 10,000 times

out_o₁ = 0.01210 (target 0.01)
out_o₂ = 0.98793 (target 0.99)
E = 0.0000044

The network has learned the mapping essentially perfectly.

Training loss over 1,000 epochs ($\eta = 0.5$)



The complete training algorithm

```
initialise all  $W, b$  randomly (small values)
```

```
repeat for each epoch:
```

```
    shuffle the training data
```

```
    for each mini-batch  $(X, Y)$ :
```

```
        # ---- FORWARD ----
```

```
         $A = X$ 
```

```
        for  $\ell = 1$  to  $L$ :
```

```
             $Z[\ell] = W[\ell] \cdot A + b[\ell]$ 
```

```
             $A = \phi(Z[\ell])$ 
```

```
        loss =  $L(A, Y)$ 
```

```
        # ---- BACKWARD ----
```

```
         $\delta = \nabla_a L \odot \phi'(Z[L])$ 
```

```
        for  $\ell = L$  down to 1:
```

```
             $dW[\ell] = \delta \cdot A[\ell-1]^T$ 
```

```
             $db[\ell] = \text{sum}(\delta)$ 
```

```
             $\delta = (W[\ell]^T \cdot \delta) \odot \phi'(Z[\ell-1])$ 
```

```
        # ---- UPDATE ----
```

```
         $W[\ell] \leftarrow W[\ell] - \eta \cdot dW[\ell]$ 
```

```
         $b[\ell] \leftarrow b[\ell] - \eta \cdot db[\ell]$ 
```

One epoch vs one step

- Epoch — one full pass over the training set.
- Iteration — one weight update, i.e. one mini-batch.
- Batch size 32, 3,200 examples $\rightarrow$ 100 iterations per epoch.

Three flavours of gradient descent

- Batch — all data per update. Stable, slow, memory-hungry.
- Stochastic (SGD) — one example per update. Fast, noisy.
- Mini-batch — 32 to 256 examples. The practical standard.

Applying It to NLP

The same two operations, now on a real language task — with a ReLU hidden layer and cross-entropy loss.

- Example 2: a sentiment classifier, forward and backward by hand
- Watching a ReLU unit die, live, in the arithmetic
- Example 3: softmax + cross-entropy for a POS tagger
- Why the output gradient simplifies to $\hat{y} - y$

A sentiment classifier with ReLU and cross-entropy

The task

Sentence : "great film, but slow"

Features : $x_1 = 1.0$ (TF-IDF of "great")

$x_2 = 0.5$ (TF-IDF of "slow")

Gold label: $y = 1$ (POSITIVE)

Architecture: $2 \rightarrow 2$ hidden (ReLU) $\rightarrow 1$ output (sigmoid), BCE loss, $\eta = 0.1$

Initial parameters

Hidden layer

$w_{11} = 0.5$ $w_{12} = -0.4$ $b_{h1} = 0.1$

$w_{21} = -0.6$ $w_{22} = 0.3$ $b_{h2} = 0.2$

Output layer

$w_5 = 0.7$ $w_6 = -0.3$ $b_o = 0.1$

What is different from Example 1 — and why it matters

- **ReLU in the hidden layer**

Gradient is 1 or 0, never a fraction. You will see one unit switch off completely.

- **Cross-entropy instead of MSE**

The output error signal collapses to the clean form $\delta = \hat{y} - y$, with no σ' factor at all.

- **A single output neuron**

Each hidden unit now has only one downstream path, so no summation is needed for δ_h .

Two hidden units, and one of them switches off

Hidden unit h_1

$$\begin{aligned} z_{h1} &= (0.5)(1.0) + (-0.4)(0.5) + 0.1 \\ &= 0.5 - 0.2 + 0.1 = 0.40 \\ h_1 &= \text{ReLU}(0.40) = 0.40 \quad \checkmark \text{ active} \end{aligned}$$

Hidden unit h_2

$$\begin{aligned} z_{h2} &= (-0.6)(1.0) + (0.3)(0.5) + 0.2 \\ &= -0.6 + 0.15 + 0.2 = -0.25 \\ h_2 &= \text{ReLU}(-0.25) = 0.00 \quad \times \text{ dead} \end{aligned}$$

Output neuron

$$\begin{aligned} z_o &= (0.7)(0.40) + (-0.3)(0.00) + 0.1 \\ &= 0.28 + 0.00 + 0.1 = 0.38 \\ \hat{y} &= \sigma(0.38) = 0.593873 \end{aligned}$$

Binary cross-entropy loss

$$\begin{aligned} L &= -[y \cdot \ln \hat{y} + (1-y) \cdot \ln(1-\hat{y})] \\ &= -[1 \cdot \ln(0.593873) + 0] \\ &= 0.521090 \end{aligned}$$

Interpretation: the model says POSITIVE with 59.4% confidence. The label is POSITIVE, so it is right — but barely. The loss of 0.521 says: correct direction, weak conviction. Backpropagation will push that confidence up.

Why sigmoid + cross-entropy gives $\delta = \hat{y} - y$

Differentiate the loss with respect to $\hat{y}$

$$L = -[y \cdot \ln \hat{y} + (1-y) \cdot \ln(1-\hat{y})]$$

$$\frac{\partial L}{\partial \hat{y}} = \frac{-y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}$$

Differentiate the sigmoid with respect to z

$$\partial \hat{y} / \partial z = \hat{y}(1 - \hat{y})$$

Multiply — the denominators cancel

$$\frac{\partial L}{\partial z} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})} \times \hat{y}(1-\hat{y}) = \hat{y} - y$$

Why this is more than elegant

With MSE + sigmoid, δ carries a $\sigma'(z)$ factor. When the model is confidently wrong ($\hat{y}$ near 0 or 1), σ' is nearly zero — so the gradient nearly vanishes exactly when the correction is most needed.

Cross-entropy removes that factor entirely. A confidently wrong prediction produces a large gradient, so the model corrects fast.

Softmax + CE gives the identical result:

$$\delta = \hat{y} - y$$

Exam-ready rule: `sigmoid+BCE` $\rightarrow \delta = \hat{y} - y$ `softmax+CE` $\rightarrow \delta = \hat{y} - y$ `sigmoid+MSE` $\rightarrow \delta = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$

Gradients — and three of them are exactly zero

Step 1 — output error signal

$$\delta_o = \hat{y} - y = 0.593873 - 1 = -0.406127$$

Negative, so the weights feeding the output will increase — pushing $\hat{y}$ up toward 1.

Step 2 — output layer gradients

$$\partial L / \partial w_5 = \delta_o \cdot h_1 = -0.406127 \times 0.40 = -0.162451$$

$$\partial L / \partial w_6 = \delta_o \cdot h_2 = -0.406127 \times 0.00 = 0.000000$$

$$\partial L / \partial b_o = \delta_o = -0.406127$$

Step 3 — hidden error signals

$$\begin{aligned} \delta_{h1} &= \delta_o \cdot w_5 \cdot \text{ReLU}'(z_{h1}) \\ &= -0.406127 \times 0.7 \times 1 = -0.284289 \end{aligned}$$

$$\begin{aligned} \delta_{h2} &= \delta_o \cdot w_6 \cdot \text{ReLU}'(-0.25) \\ &= -0.406127 \times (-0.3) \times 0 = 0.000000 \end{aligned}$$

Step 4 — hidden layer gradients

$$\partial L / \partial w_{11} = \delta_{h1} \cdot x_1 = -0.284289 \times 1.0 = -0.284289$$

$$\partial L / \partial w_{12} = \delta_{h1} \cdot x_2 = -0.284289 \times 0.5 = -0.142144$$

$$\partial L / \partial w_{21} = \delta_{h2} \cdot x_1 = 0 \times 1.0 = 0.000000$$

$$\partial L / \partial w_{22} = \delta_{h2} \cdot x_2 = 0 \times 0.5 = 0.000000$$

The dying ReLU, caught in the act

Because z_{h2} was negative, $\text{ReLU}'(z_{h2}) = 0$. That single zero wipes out δ_{h2} , and with it the gradients for w_{21} , w_{22} and b_{h2} . Those three parameters do not move on this example. If z_{h2} stays negative for every training example, they will never move again — the unit is permanently dead. Leaky ReLU would have left a gradient of $\alpha = 0.01$ instead of 0, keeping it alive.

Applying $\eta = 0.1$ and re-running the network

parameter	before	gradient	after
w_5	0.700000	-0.162451	0.716245
w_6	-0.300000	0.000000	-0.300000
b_o	0.100000	-0.406127	0.140613
w_{11}	0.500000	-0.284289	0.528429
w_{12}	-0.400000	-0.142144	-0.385786
w_{21}	-0.600000	0.000000	-0.600000
w_{22}	0.300000	0.000000	0.300000
b_{h1}	0.100000	-0.284289	0.128429
b_{h2}	0.200000	0.000000	0.200000

Rows in red did not move — the dead unit's parameters.

Re-run the forward pass

$$\begin{aligned}
 z_{h1} &= 0.528429 + (-0.385786)(0.5) + 0.128429 \\
 &= 0.463965 \\
 h_1 &= 0.463965 \quad h_2 = 0 \text{ (still dead)} \\
 z_o &= (0.716245)(0.463965) + 0.140613 \\
 &= 0.472921 \\
 \hat{y} &= \sigma(0.472921) = 0.616076
 \end{aligned}$$

$$\begin{aligned}
 L \text{ before} &= 0.521090 \\
 L \text{ after} &= 0.484385 \\
 \Delta L &= -0.036705 \quad (-7.0\%)
 \end{aligned}$$

Confidence rose from 59.4% to 61.6% in a single step — in the right direction, using only half the network.

Softmax + cross-entropy for a POS tagger

Forward

Word: "book" Classes: NOUN, VERB, ADJ

Logits $z = [2.0 , 1.0 , 0.1]$

Gold $y = [1 , 0 , 0]$ (NOUN)

$\hat{y} = \text{softmax}(z) = [0.6590, 0.2424, 0.0986]$

$L = -\ln(\hat{y}_{\text{NOUN}}) = -\ln(0.6590) = 0.4170$

Backward — one subtraction

$$\delta = \hat{y} - y$$

$$\delta_{\text{NOUN}} = 0.6590 - 1 = -0.3410$$

$$\delta_{\text{VERB}} = 0.2424 - 0 = 0.2424$$

$$\delta_{\text{ADJ}} = 0.0986 - 0 = 0.0986$$

$$\text{Sum of } \delta = 0.0000 \quad \checkmark \quad (\text{always true})$$

Reading the error signals

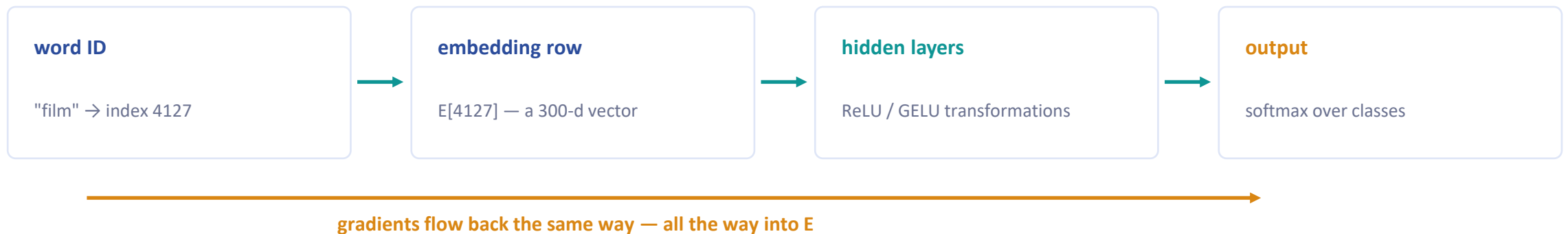
- Negative δ on the correct class $\rightarrow$ push its logit UP.
- Positive δ on wrong classes $\rightarrow$ push their logits DOWN.
- The magnitudes are the probabilities themselves — the more mass on a wrong class, the harder it is pushed down.

Scaling up to a real tagger

Replace 3 classes with 45 Penn Treebank tags, or with a 50,000-word vocabulary for language modelling. The vector gets longer; the mathematics on this slide does not change by a single symbol. δ is still $\hat{y} - y$.

Backpropagating into word embeddings

In NLP the input layer is usually an embedding lookup: the word ID selects one row of an embedding matrix E of shape $(V \times d)$, where V is the vocabulary size and d the embedding dimension. That lookup is just a matrix multiply by a one-hot vector — so gradients flow through it exactly like any other layer.



The sparsity trick

Only the rows for words actually in the batch receive a non-zero gradient. A 50,000-word embedding table updates just a few hundred rows per step — which is why the operation stays cheap.

Freeze or fine-tune?

Freeze pretrained embeddings when your dataset is small — fine-tuning would overfit. Fine-tune when you have plenty of task-specific data, or when the domain is unusual (legal, clinical, code).

Learning rate, initialisation, and what usually goes wrong

Choosing the learning rate η

too small ($1e-6$) training crawls; may never converge

too large (10) loss oscillates or explodes to NaN

about right ($1e-3$) loss falls steadily and levels off
Standard practice: Adam with $\eta = 1e-3$, or $2e-5$ to $5e-5$ when fine-tuning a transformer.

Weight initialisation

- All zeros — fatal. Every neuron computes the same thing and receives the same gradient forever (the symmetry problem).
- Xavier / Glorot — variance $1/n_{in}$. Use with tanh or sigmoid.
- He / Kaiming — variance $2/n_{in}$. Use with ReLU. The 2 compensates for half the units being zeroed.

● Loss is NaN

Learning rate too high, or $\log(0)$ in cross-entropy. Lower η ; clamp probabilities away from 0 and 1.

● Loss does not move

η too small, dead ReLUs, or a bug in the gradient. Check with gradient checking.

● Train loss ↓, val loss ↑

Overfitting. Add dropout, weight decay, or early stopping; get more data.

● Loss oscillates wildly

Batch too small or η too high. Increase batch size or add a learning-rate schedule.

The seven mistakes students make most often

1

Forgetting to sum over paths

A hidden neuron that feeds three outputs has THREE error contributions. Add them all before multiplying by ϕ' .

2

Using updated weights mid-pass

Compute every gradient with the old parameters, then update everything at once.

3

Dropping the $\phi'(z)$ factor

δ always includes the activation derivative — except when sigmoid/softmax cancels against cross-entropy.

4

Evaluating ϕ' at the wrong point

σ' is evaluated at net (the pre-activation), even though the shortcut formula $\text{out}(1-\text{out})$ uses the output value.

5

Sign errors in the update

The rule is $w \leftarrow w - \eta \cdot \text{grad}$. Subtract. Adding it climbs the loss surface.

6

Mismatched matrix shapes

In the backward pass W is TRANSPOSED. If shapes do not conform, you have the orientation wrong.

7

Reporting the loss after updating

Compare like with like: the loss that produced the gradient is the pre-update loss.

Debugging tip: gradient checking. Compare your analytic gradient against $(L(w+\epsilon) - L(w-\epsilon)) / 2\epsilon$ with $\epsilon = 1e-7$. They should agree to about six decimal places.

Practice set — attempt before looking at the next slide

Q1 Single neuron

$x = [1, 2]$, $w = [0.3, -0.2]$, $b = 0.5$.

Compute z , then the output under sigmoid, tanh and ReLU.

Q2 Softmax + loss

Logits $z = [1.0, 2.0, 3.0]$.

Compute the softmax distribution, then the cross-entropy loss if the true class is the third.

Q3 Output gradient

A sigmoid output gives $\hat{y} = 0.8$; target $t = 0.5$; incoming activation $a = 0.6$; $w = 0.4$; $\eta = 0.1$.

Using MSE, find δ , $\partial E / \partial w$ and the updated w .

Q4 Hidden delta

Hidden unit h (sigmoid, $\text{out}_h = 0.7$) feeds two outputs with $\delta_{o1} = 0.2$ and $\delta_{o2} = -0.1$, through weights 0.5 and 0.8.

Find δ_h .

Q5 Vanishing gradient

A network has 4 sigmoid hidden layers.

What is the largest possible product of the activation derivatives along one path?

Q6 Conceptual

Explain in two sentences why replacing ReLU with the identity function in every hidden layer would make a 10-layer network no more powerful than logistic regression.

Worked answers

A1

$$z = (0.3)(1) + (-0.2)(2) + 0.5 = 0.4$$

$$\sigma(0.4) = 0.5987 \quad \tanh(0.4) = 0.3799 \quad \text{ReLU}(0.4) = 0.40$$

A2

$$e = [2.7183, 7.3891, 20.0855], \text{ sum} = 30.1929$$

$$\hat{y} = [0.0900, 0.2447, 0.6652] \quad L = -\ln(0.6652) = 0.4076$$

A3

$$\delta = (0.8 - 0.5) \times 0.8 \times 0.2 = 0.048$$

$$\partial E / \partial w = 0.048 \times 0.6 = 0.0288 \quad w^+ = 0.4 - 0.1(0.0288) = 0.39712$$

A4

$$\Sigma = (0.2)(0.5) + (-0.1)(0.8) = 0.10 - 0.08 = 0.02$$

$$\sigma' = 0.7 \times 0.3 = 0.21 \quad \delta_h = 0.02 \times 0.21 = 0.0042$$

A5

$$(0.25)^4 = 0.00390625$$

Under 0.4% of the signal survives – the early layers barely learn.

A6

Composing linear maps yields another linear map: $W_{10} \cdots W_1 x + b' = W'x + b'$.
The 10 layers collapse into one, so the decision boundary stays a single hyperplane.

What to take away

- **A neuron is two operations**

A weighted sum followed by a non-linearity. Everything else is repetition of this.

- **Activations make depth meaningful**

Without ϕ , any number of layers collapses into a single linear map.

- **Feedforward is $Wa + b$, then ϕ**

Applied layer by layer until a prediction appears at the end.

- **Backpropagation is the chain rule**

Three factors: how wrong the output is, how responsive the activation is, and what the incoming signal was.

- **δ is the reusable quantity**

Compute it once per neuron; every weight and bias gradient at that neuron follows from it.

- **Loss and activation are a pair**

Sigmoid/softmax with cross-entropy gives $\delta = \hat{y} - y$. Choose them together, not separately.

Next: from these fundamentals to word embeddings, recurrent networks, and the transformer.

References and recommended reading

Textbooks

- Jurafsky & Martin — Speech and Language Processing, 3rd ed., Chapter 7 (Neural Networks)
- Goodfellow, Bengio & Courville — Deep Learning, Chapters 6 and 8
- Eisenstein — Introduction to Natural Language Processing, Chapter 3

Foundational papers

- Rumelhart, Hinton & Williams (1986) — Learning representations by back-propagating errors
- Glorot & Bengio (2010) — Understanding the difficulty of training deep feedforward networks
- He et al. (2015) — Delving deep into rectifiers (He initialisation, PReLU)
- Hendrycks & Gimpel (2016) — Gaussian Error Linear Units

Practice

- Implement Example 1 in NumPy without any framework — verify every number in this deck
- Then reimplement it in PyTorch and confirm autograd returns the same gradients
- Extend Example 2 to a real dataset (IMDB or SST-2) with a bag-of-embeddings input

Gradient checking is the best debugging tool you will learn this term — use it every time you write a backward pass by hand.